

PROJETO.....

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

VICTOR HUGO DE MELO RIBEIRO - RA: 26029726

GILDO SOUZA JUNIOR - RA: 26029658

KAINAN DE SOUZA ALVES - RA: 26029656

RAISSA SOUSA NUNES - RA: 26029670

KAUÊ LIMA CERQUEIRA - RA: 26029835

São Paulo

2025

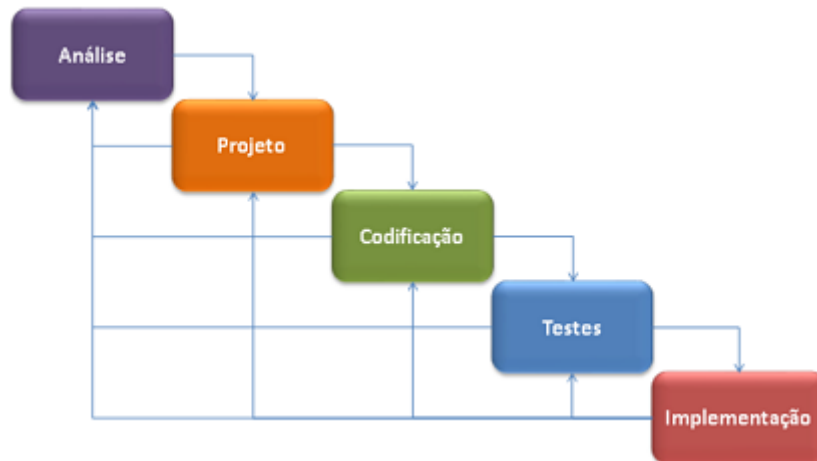
Sumário

1 INTRODUÇÃO	3
1.1 – Processo de desenvolvimento de Software	3
1.2 Apresentação	3
2. DESIGN SPRINT – Ideação e prototipação do desafio	3
2.1 Desafio	3
2.2 Entender Mapear	4
2.2.1 – Entendimento e observação. COMO PODEMOS?	4
2.2.2 – Pesquisa Desk	4
2.3 Ideação – desenho da solução (trilha do usuário)	4
2.4 Prototipagem	4
3. REQUISITOS DE SISTEMA	4
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE	4
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE	5
4. DIAGRAMAS DE UML	6
4.1 CASOS DE USO	6
4.2 DIAGRAMA DE CLASSE	6
5. Especificação estrutura de uma rotina do sistema	6
5.1 – Apresentar a especificação de acordo com o modelo abaixo	6
5.2 – Apresentar a codificação da rotina	7
6. ARQUITETURA DO SISTEMA	7
6.1. Visão Geral do Sistema	7
6.2. Arquitetura Geral	7
6.3. Componentes Arquiteturais e Camadas	8
6.4. Diagrama da Arquitetura	8
6.5. Organização de Pastas e Módulos	9
6.6. Integrações Externas (se houver)	10
6.7. Ambiente e Implantação	10
8. REFERÊNCIAS BIBLIOGRÁFICAS	10

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

Exemplo (substituir) - o que foi realizado em sala no Draw.io, escolha um por equipe



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produto e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote. Tendo.....

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI – Desenvolver um sistema digital lúdico e educativo que utilize perguntas e regras de jogo armazenadas em Banco de Dados para promover a familiarização do usuário com a Arcor, suas marcas, linhas e produtos, integrando conhecimentos das Unidades Curriculares do 1 Semestre de Análise e Desenvolvimento de Sistemas.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

Público juvenil. Definido

Movimentação Scriptada. Definido

Ilhas temáticas das Marcas. Definido

Como podemos definir o sistema de pontuação do jogo?

Como podemos definir a ordem e o sistema de dificuldade?

Como podemos definir o sistema de bonificação?

Como podemos refinar o gráfico no pós?

Como podemos prender a atenção do jogador?

2.2.2 – Pesquisa Desk

Arte similar de undertale

2.3 Ideação – desenho da solução (trilha do usuário)

Se tivesse superpoderes? Poder de dominar o tempo e criar um jogo mais qualificado

Se eu tivesse muito dinheiro? Contrataria um studio consolidado pra fazer o jogo/quiz

Se eu não tivesse dinheiro nenhum? Jogo de tabuleiro/manual

Se fosse o Dick Vigarista e quisesse boicotar? Hackeava o jogo e pulava etapas

Crazy 8 ideias: Mundo aberto -

1 Tartaruga matando os boss com foice

2 Chocolate faz mal pra saúde

3 Tartaruga matando os plásticos

4 Top nacional ganhar a empresa inteira

5 Propaganda infantil incentivando consumo de chocolate

6 Tortuguinha pisando nos concorrentes no final do jogo

7 Tortuguinha lutando pelos seus direitos

8 Arena de guerra das tortuguinhas



2.4 Prototipagem

3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Cadastrar usuário
Descrição	Permite que um novo jogador crie uma conta no site para poder jogar e ter seu progresso registrado.
Entradas	Nome, email, senha e confirmação de senha.

Fonte	Jogador, por meio do formulário de cadastro.
Saídas	Mensagem de cadastro realizado e redirecionamento para a tela de login, ou mensagem de erro indicando o problema.
Ação	1. O jogador acessa a tela de cadastro e preenche os campos. 2. O sistema verifica se todos os campos foram preenchidos, se o email tem formato válido e se a senha e a confirmação são iguais. 3. O sistema verifica se o email já está cadastrado. 4. O sistema grava o usuário, com a senha criptografada e a data de cadastro. 5. O sistema exibe a confirmação e leva o jogador à tela de login.

RFS02	
Função	Autenticar usuário
Descrição	Permite que o jogador cadastrado entre no sistema e saia dele, garantindo que cada pessoa acesse apenas as próprias partidas.
Entradas	Email e senha.
Fonte	Jogador, por meio do formulário de login. Dados do usuário armazenados no banco de dados.
Saídas	Acesso à tela inicial do jogo, ou mensagem de email ou senha inválidos.
Ação	1. O jogador informa email e senha na tela de login. 2. O sistema busca o usuário pelo email e compara a senha informada com a senha armazenada. 3. Se os dados estiverem corretos, o sistema inicia a sessão do usuário e exibe a tela inicial. 4. Se estiverem incorretos, o sistema exibe a mensagem de erro e mantém o jogador na tela de login. 5. Ao escolher sair, o sistema encerra a sessão e retorna à tela de login.

RFS03	
Função	Iniciar partida
Descrição	Permite que o jogador autenticado comece uma nova partida do jogo ou continue uma partida em andamento.
Entradas	Comando “Novo Jogo” ou “Continuar”
Fonte	Jogador autenticado. Dados da partida e da primeira ilha armazenados no banco de dados.
Saídas	Partida criada ou retomada, com a tartaruga posicionada na ilha atual, exibindo vida e pontuação.
Ação	1. O sistema confirma que o usuário está autenticado. 2. Se houver partida em andamento, o sistema oferece continuar ou começar outra. 3. Em um novo jogo, o sistema cria a partida com data de início, status “Em andamento”, vida atual inicial e pontuação zero. 4. O sistema associa a partida à primeira ilha (menor ordem). 5. O sistema exibe a ilha, a tartaruga, a vida e a pontuação.

RFS04	
Função	Percorrer as ilhas
Descrição	Controla a viagem da tartaruga de ilha em ilha, respeitando a ordem e a dificuldade (fácil, média ou difícil). Uma ilha só é liberada quando o jogador atinge os acertos necessários da ilha anterior.
Entradas	Escolha do jogador de avançar para a próxima ilha.
Fonte	Jogador. Dados das ilhas (descrição, dificuldade, ordem e acertos necessários) e progresso da partida no banco de dados.
Saídas	Ilha exibida com descrição e dificuldade, aviso de ilha bloqueada ou mensagem de fim de jogo na última ilha.
Ação	1. O jogador solicita avançar para a próxima ilha. 2. O sistema verifica se os acertos necessários da ilha atual foram atingidos.

	3. Se sim, o sistema registra a passagem da partida pela nova ilha e exibe sua descrição e dificuldade. 4. Se não, o sistema informa que a ilha ainda está bloqueada e quantos acertos faltam. 5. Ao concluir a última ilha, o sistema registra
--	--

RFS05	
Função	Enfrentar inimigos
Descrição	Em cada ilha a tartaruga enfrenta inimigos de acordo com a dificuldade. O resultado do combate altera a vida atual e a pontuação da partida.
Entradas	Ação de combate escolhida pelo jogador (por exemplo, atacar).
Fonte	Jogador. Dados da partida (vida atual e pontuação) e da ilha onde a tartaruga está.
Saídas	Resultado do combate, vida atual e pontuação atualizadas e, se a vida chegar a zero, mensagem de derrota.
Ação	1. O sistema apresenta um inimigo compatível com a dificuldade da ilha. 2. O jogador executa a ação de combate. 3. O sistema calcula o dano causado e o dano recebido e atualiza a vida atual da tartaruga. 4. Se o inimigo for derrotado, o sistema soma pontos à partida e libera o quiz da ilha. 5. Se a vida atual chegar a zero, o sistema encerra a partida com status “Derrotada”, registra a data de fim e oferece iniciar um novo jogo.

RFS06	
Função	Responder ao quiz
Descrição	Apresenta as perguntas do quiz da ilha, registra a alternativa escolhida pelo jogador, informa se está correta e atualiza os acertos e a pontuação. Os acertos necessários permitem concluir a ilha.

Entradas	Alternativa escolhida pelo jogador.
Fonte	Jogador. Perguntas e alternativas da ilha armazenadas no banco de dados.
Saídas	Resultado da resposta (correta ou incorreta), explicação da pergunta, acertos e pontuação atualizados e aviso de ilha concluída, quando for o caso.
Ação	1. O sistema seleciona uma pergunta da ilha atual e exibe o enunciado e as alternativas. 2. O jogador escolhe uma alternativa. 3. O sistema registra a escolha vinculada à partida. 4. O sistema verifica se a alternativa escolhida é a correta. 5. O sistema exibe o resultado e a explicação e atualiza os acertos e a pontuação. 6. Quando o número de acertos necessários é atingido, o sistema marca a ilha como concluída e libera a próxima (RFS04).

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Usabilidade
Descrição	A interface do jogo deve ser fácil de entender, com textos em português e retorno visual claro para ações como acertar, errar e perder vida, de modo que o jogador use o site sem ajuda externa.
Entradas	Interações do jogador com a tela (cliques, toques e teclado).
Fonte	Jogador, usando o site em um navegador.
Saídas	Telas organizadas, mensagens claras e retorno visual imediato para cada ação (acerto, erro, vida e pontuação).
Ação	1. O sistema apresenta menus, botões e textos em português, com nomes claros. 2. A cada ação do jogador, o sistema exibe um retorno visual, como cor, mensagem ou animação simples. 3. Vida, pontuação e ilha atual ficam sempre visíveis durante a partida. 4. Mensagens de erro explicam o problema e indicam como corrigi-lo. 5. Critério de verificação: um usuário novo consegue se cadastrar e iniciar uma partida sem instruções adicionais.

RFS02	
Função	Desempenho
Descrição	O sistema deve responder rapidamente às ações do jogador, para que o jogo seja fluido e o quiz não tenha esperas longas.
Entradas	Solicitações do jogador, como abrir uma tela, carregar uma pergunta e enviar uma resposta.
Fonte	Jogador, por meio do navegador. Dados armazenados no banco de dados.
Saídas	Telas e perguntas carregadas em até 3 segundos e resultado da resposta exibido em até 1 segundo, em conexão comum de internet.
Ação	1. O sistema recebe a solicitação do jogador. 2. O sistema consulta no banco de dados apenas os dados necessários para aquela tela (por exemplo, a pergunta atual e suas alternativas). 3. O sistema processa a solicitação e devolve a resposta ao navegador. 4. Imagens e demais arquivos do jogo são mantidos leves para não atrasar o carregamento. 5. Critério de verificação: os tempos acima são respeitados em testes com uma conexão ruins.

RFS03	
Função	Segurança
Descrição	As senhas devem ser armazenadas criptografadas, e cada usuário deve acessar somente as próprias partidas. Os dados enviados devem ser validados para evitar ataques como a injeção de SQL, e a correção das respostas do quiz deve ser feita no servidor.
Entradas	Dados de cadastro e login (email e senha) e alternativas escolhidas no quiz.
Fonte	Jogador, por meio dos formulários e das telas do jogo.
Saídas	Acesso liberado apenas a usuários autenticados, senhas protegidas no banco de dados e respostas verificadas com segurança.
Ação	1. Ao cadastrar um usuário, o sistema transforma a senha em hash antes de gravá-la no banco de dados. 2. O sistema valida e trata todos os dados recebidos antes de usá-los em consultas ao banco de dados. 3. Páginas internas e dados de partidas só são acessados com sessão de usuário válida. 4. O sistema confere se a alternativa escolhida é a correta no servidor, sem enviar essa informação ao navegador antes da resposta. 5. Critério de verificação: nenhuma senha aparece em texto puro no banco de dados e páginas internas não abrem sem login.

RFS04	
Função	Compatibilidade e Portabilidade
Descrição	O site deve funcionar nas versões recentes dos principais navegadores e se adaptar a diferentes tamanhos de tela, como computador, tablet e celular.
Entradas	Acesso do jogador ao site por navegador e dispositivo variados.
Fonte	Jogador, usando computador, tablet ou celular.
Saídas	Jogo exibido e funcionando corretamente, com telas legíveis e utilizáveis em qualquer tamanho de tela.
Ação	1. O sistema usa um layout responsivo, que ajusta tamanhos e posições dos elementos na tela

RFS04	
Função	Compatibilidade e Portabilidade
Descrição	O site deve funcionar nas versões recentes dos principais navegadores e se adaptar a diferentes tamanhos de tela, como computador, tablet e celular.
	2. O sistema usa apenas recursos web com suporte nos navegadores 3. Textos, botões e alternativas do quiz

RFS05	
Função	Confiabilidade
Descrição	O progresso da partida (ilha atual, vida, pontuação e respostas) deve ser salvo automaticamente, e o banco de dados deve manter a consistência entre as informações, sem perguntas sem alternativas ou respostas sem partida.
Entradas	Ações do jogador durante a partida, como responder uma pergunta, vencer um inimigo ou concluir uma ilha.
Fonte	Jogador. Dados da partida armazenados no banco de dados.
Saídas	Progresso salvo e partida retomada do ponto em que parou, com dados consistentes.
Ação	1. A cada resposta, combate ou conclusão de ilha, o sistema grava o progresso no banco de dados. 2. O banco de dados usa chaves primárias e estrangeiras para manter o vínculo entre usuário, partida, ilha, pergunta e alternativa. 3. Se a conexão for interrompida ou o site for fechado, o jogador pode retomar a partida em andamento. 4. Se uma gravação falhar, o sistema informa o erro e não deixa dados incompletos salvos. 5. Critério de verificação: ao fechar e reabrir o site, o jogador retoma a partida perdendo no máximo a última ação.

4.

RFS06	
Função	Manutenibilidade
Entradas	Novos registros de ilhas, perguntas e alternativas, e alterações no código feitas pela equipe de desenvolvimento.
Fonte	Equipe de desenvolvimento e administrador do sistema.
Saídas	Conteúdo do jogo atualizado e funcionando sem necessidade de reescrever o código.

RFS06	
Função	Manutenibilidade
Ação	1. O código é dividido em partes com responsabilidades claras (por exemplo, telas, regras do jogo e acesso ao banco de dados). 2. O código recebe nomes claros e comentários nos trechos principais. 3. Ilhas, perguntas e alternativas ficam em tabelas do banco de dados, e não escritas diretamente no código. 4. Para ampliar o jogo, a equipe apenas insere novos registros nessas tabelas. 5. Critério de verificação: uma nova pergunta ou ilha pode ser incluída apenas inserindo registros no banco de dados.

4.

DIAGRAMAS DE UML

4.1 CASOS DE USO

Apresentar 3 casos de uso do sistema

4.2 DIAGRAMA DE CLASSE

Apresentar 1 diagrama de classe do sistema

5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

Sistema/Software

Função

Descrição

Entradas

Fonte

Saídas

Destino

Ação

•

Requisitos

Pré-condição

Pós-condições

Efeitos colaterais

5.2 – Apresentar a codificação da rotina

Apresente em pseudocódigo, Python ou outra linguagem

6. ARQUITETURA DO SISTEMA

Para a arquitetura do sistema, considere o template abaixo:

Template (esquema de arquitetura) para orientar a elaboração de um **Projeto de Arquitetura de Sistema Web**, cobrindo desde a camada de apresentação até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema:**
- **Objetivo do Sistema:** (Breve descrição da finalidade do sistema)
- **Público-Alvo:**
- **Principais Funcionalidades:**

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Exemplo: Arquitetura em Camadas (Layered Architecture) com separação clara entre apresentação, lógica de negócios e persistência.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias:** (Ex.: HTML5, CSS3, JavaScript, React/Vue/Angular)

- **Responsabilidades:**

- Interface com o usuário
- Validação inicial de dados
- Comunicação com a API via HTTP/JSON

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias:** (Ex.: Node.js, Django, Spring Boot, Flask, Laravel)

- **Responsabilidades:**

- Implementação da lógica de negócio
- Autenticação/autorização
- Comunicação com a camada de dados
- Exposição de endpoints RESTful/GraphQL

6.3.3. Camada de Domínio (Negócio)

- **Descrição:**

- Contém as regras de negócio e os modelos de domínio
- Independente das demais camadas

- **Componentes:**

- Entidades
- Casos de uso
- Regras de validação

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias:** (Ex.: PostgreSQL, MySQL, MongoDB, SQLite)

- **Responsabilidades:**

- Armazenamento e recuperação de dados
- Scripts de criação e migração de banco

6.4. Diagrama da Arquitetura

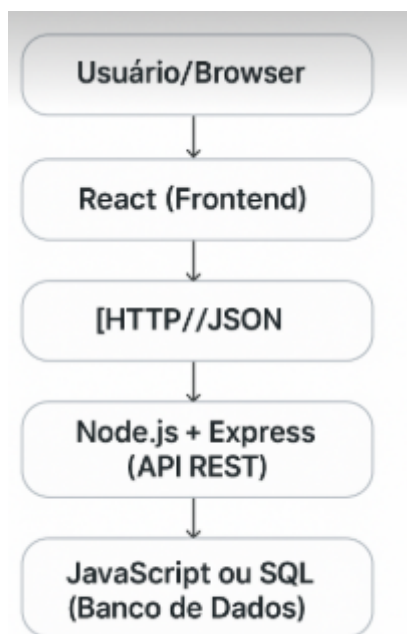
Inserir um **diagrama estrutural** (ex.: em draw.io, Lucidchart ou outro) mostrando:

- Fluxo entre frontend, backend, banco de dados
- Componentes principais
- APIs e serviços envolvidos

- Gateways, autenticação, filas de mensagens (se houver)

Exemplo textual de uma arquitetura de sistema web.

[Usuário/Browser]
 |
 React (Frontend)
 |
 [HTTP/JSON]
 |
 Node.js + Express (API REST)
 |
 JavaScript ou SQL
 |
 PostgreSQL (Banco de Dados)
 Diagrama:



6.5. Organização de Pastas e Módulos

Descrever como o código será organizado no repositório:

```

  /frontend
    /components/
    /pages/
    /services/
  /backend
    /controllers/
    /services/
    /models/
    /routes/
    /config/
  
```

 /database
 schema.sql
 migrations/

6.6. Integrações Externas (se houver)

- APIs de terceiros (pagamentos, mapas, redes sociais)
- Serviços de autenticação (OAuth2, Firebase, etc.)
- Serviços de envio (email, notificações)

6.7. Ambiente e Implantação

- **Ambientes:** desenvolvimento, homologação, produção
- **Contêineres:** Docker/Docker Compose (se usado)
- **CI/CD:** GitHub Actions, GitLab CI, Jenkins
- **Hospedagem:** Heroku, Vercel, AWS, GCP, Azure

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.